

BIG DATA REPORT

In fulfillment of one of the final exams for the Big Data course, taught by:

Ronny Juwono, S.Pd., M.T.



Created By:

Nadira Khumaira Putri

(012202400029)

**FACULTY OF ARTIFICIAL INTELLIGENCE AND SMART
MANUFACTURING**

PRESIDENT UNIVERSITY

2026

INTRODUCTION

This capstone project was completed as part of a Big Data course workshop provided by Mattel, focused on building a containerized, multi-pipeline data orchestration system using Docker and Dagster.

Docker is a containerization platform that packages an application together with all its dependencies code, runtime, libraries, and system tools into a single, isolated unit called a container. Containers ensure that a pipeline behaves identically regardless of where it runs, which is critical in big data environments where multiple data pipelines, each with different dependencies and language versions, often need to run side by side without conflicting with one another.

Dagster is an open-source data orchestration framework built specifically for defining, scheduling, and monitoring data pipelines as software. Instead of treating a pipeline as a sequence of loosely connected scripts, Dagster models each unit of data (a table, a file, a trained model) as an "asset" with explicit dependencies on other assets, forming a directed graph. This makes data lineage, scheduling, and data-quality validation (via asset checks) first-class, observable parts of the pipeline rather than an afterthought.

Combining Docker and Dagster reflects a common real-world big data architecture pattern: each data source or workload runs in its own container (its own dependencies, its own failure domain), while a central Dagster webserver and daemon orchestrate all containers together through a shared configuration (workspace.yaml), and all pipelines converge on a single shared destination, typically a data warehouse. This mirrors how production big data systems ingest data from many independent internal or external systems into one central store for analytics.

This workshop is directly relevant to the Big Data course because it demonstrates, hands-on, the two foundational concerns of any big data system: (1) how to isolate and scale independent data-processing units using containerization, and (2) how to orchestrate, schedule, and monitor the flow of data between those units reliably the same concerns that underpin large-scale data engineering platforms in industry.

The full implementation, including the exercises and the Track B capstone pipeline, is available at: <https://github.com/nadiraaa123/dagster-workshop-multi>

ASSIGNMENTS

Repository: <https://github.com/nadiraaa123/dagster-workshop-multi>

Scope: pipeline_products and pipeline_fx

1. Background

This workshop utilizes Dagster with a multi-container architecture, where each pipeline (pipeline_products, pipeline_fx, pipeline_ml) runs as a separate container and connects via a shared data warehouse (Postgres). The exercises aim to practice three core Dagster concepts: creating downstream assets, reading data across containers, and implementing data quality checks.

2. Exercise (1) — Asset `top\_selling\_products`

- File: `pipeline\_products/main.py`
- Objective: Create a new asset displaying the 5 products with the highest total sales quantity, derived from the `raw\_orders` and `raw\_products` assets.
- Implementation: This asset merges `raw\_orders` and `raw\_products` data based on the `product\_id` column, sums the quantity per product (using `groupby...sum`), sorts the results in descending order, and selects the top 5. The asset is registered in `Definitions` to ensure it is connected within the Dagster dependency graph.

```
# TODO(exercise-1): add a `top_selling_products` asset downstream of
# raw_orders and raw_products (join on product_id, sum quantity) - see
# docs/exercises.md
@asset
def top_selling_products(raw_orders: pd.DataFrame, raw_products: pd.DataFrame) -> pd.DataFrame:
    merged = raw_orders.merge(raw_products, on="product_id", how="inner")
    totals = (
        merged.groupby("product_id")["quantity"]
        .sum()
        .reset_index()
        .sort_values("quantity", ascending=False)
        .head(5)
    )
    return totals.merge(raw_products[["product_id", "name"]], on="product_id", how="left")
```

```
defs = Definitions(
    assets=[raw_products, raw_orders, products_table, orders_table, top_selling_products],
    asset_checks=[raw_orders_no_invalid_quantity],
    jobs=[refresh_products_job],
    schedules=[refresh_products_daily],
)
```

- Verification: The asset is successfully materialized via the Dagster UI after its upstream assets (`raw\_orders`, `raw\_products`) have been materialized, following the dependency graph order.

Assets > raw_orders

Overview Events Checks Lineage

Status

Latest materialization: 28 Jul, 23:04 (Materialized)

Check results: 1 (Pass)

Recent updates: Showing all 2 updates

24/7/2026 28/7/2026

Description

No description found. You can add a description to any asset by adding a 'description' argument to it. Learn more

Metadata

Filter metadata keys

Key	Timestamp	Value
path	28 Jul, 23:04	/opt/dagster/dagster_home/storage/raw_orders

Lineage

Upstream assets: raw_orders, raw_products

Downstream assets: orders_table, top_selling_products

View in graph

Asset	Materialized
raw_orders	28 Jul, 23:04
raw_products	28 Jul, 23:04

3. Exercise (2) — Asset `orders\_in\_eur`

- File: `pipeline\_fx/main.py`
- Objective: Create an asset that reads cross-pipeline data (`orders` and `products` from `pipeline\_products`) and converts it to EUR using `exchange\_rates` data from `pipeline\_fx`.
- Implementation: Since `orders` and `products` are written by a different container (`pipeline\_products`), this asset does not use a direct Dagster dependency on those assets; instead, it reads directly from the shared Postgres warehouse using a connection (`db.get\_engine()` + `pd.read\_sql`). This demonstrates how two independent pipelines can remain connected via a single destination database without direct inter-container dependencies. The total order value (quantity $\times$ price) is converted to EUR using the rate from the `exchange\_rates` table.

```
# TODO(exercise-2): add an `orders_in_eur` asset that reads the `orders` and
# `products` tables written by pipeline_products, joins them with
# `exchange_rates_table`, and converts order totals to EUR - see
# docs/exercises.md. This is a cross-container exercise: pipeline_fx and
# pipeline_products both write to the same warehouse Postgres, so this asset
# can just read those tables directly with the Warehouse engine.
@asset(deps=[exchange_rates_table])
def orders_in_eur() -> pd.DataFrame:
    engine = db.get_engine()
    orders = pd.read_sql("SELECT * FROM orders", engine)
    products = pd.read_sql("SELECT * FROM products", engine)
    rates = pd.read_sql("SELECT * FROM exchange_rates", engine)

    merged = orders.merge(products, on="product_id", how="inner")
    merged["total_usd"] = merged["quantity"] * merged["price"]
    eur_rate = rates.loc[rates["quote_currency"] == "EUR", "rate"].iloc[0]
    merged["total_eur"] = merged["total_usd"] * eur_rate

    return merged[["order_id", "customer_id", "product_id", "quantity", "total_usd", "total_eur"]]
```

```
defs = Definitions(
    assets=[raw_exchange_rates, exchange_rates_table, orders_in_eur],
    jobs=[refresh_fx_job],
    schedules=[refresh_fx_daily],
)
```

```
from dagster import Definitions, ScheduleDefinition, asset, define_asset_job
```

Don't forget to add the library

- Verification: The asset is successfully materialized, producing a combined table containing order values in both USD and EUR.

 orders_in_eur	 pipeline_fx  default	 Materialized · 28 Jul, 22:45
--	---	---

4. Exercise (3) — Asset Check `raw\_orders\_no\_invalid\_quantity`

- File: `pipeline\_products/main.py`
- Objective: Add a data quality check to the `raw\_orders` asset to detect data rows with invalid quantity values (≤ 0).
- Implementation: Uses the `@asset\_check` decorator attached to the `raw\_orders` asset. The check filters for rows where `quantity <= 0` and returns an `AssetCheckResult` indicating whether the check passed, accompanied by `invalid\_row\_count` metadata for result transparency. The check is registered via the `asset\_checks` parameter in `Definitions`.

```
# TODO(exercise-3): add an @asset_check on raw_orders that fails if any row
# has quantity <= 0 - see docs/exercises.md

@asset_check(asset=raw_orders)
def raw_orders_no_invalid_quantity(raw_orders: pd.DataFrame) -> AssetCheckResult:
    invalid_rows = raw_orders[raw_orders["quantity"] <= 0]
    return AssetCheckResult(
        passed=invalid_rows.empty,
        metadata={"invalid_row_count": len(invalid_rows)},
    )
```

- Verification: The check executed successfully with a status of "Passed" and `invalid\_row\_count: 0`, indicating that all `raw\_orders` data is valid (no negative or zero quantities).

Assets > raw_orders

Overview Events Checks Lineage

Checks (1) Execute all

raw_orders_no_invalid_quantity

Filter checks

raw_orders_no_invalid_quantity
Succeeded

Overview Execution history Automation history

About

No description provided

Latest execution

Evaluation result	Timestamp	Target materialization
Passed	28 Jul, 23:03	28 Jul, 23:04

Metadata

invalid_row_count	0
-------------------	---

5. Challenges and Solutions During Implementation

- The asset check initially did not appear in the main Dagster UI graph; it turned out that asset checks are not displayed as separate nodes but are instead located under the "Checks" tab on the specific asset's page (`raw\_orders`).
- A `FileNotFoundError` occurred when materializing a downstream asset (`top\_selling\_products`) and running the check separately; this was caused by the upstream assets (`raw\_orders`, `raw\_products`) not yet being saved to storage within the same run. The solution was to materialize the entire asset chain, from upstream to downstream, in a single process/run, rather than executing downstream assets or checks in isolation.

6. Conclusion

- All three exercises were successfully completed and verified via the Dagster UI: the `top\_selling\_products` and `orders\_in\_eur` assets were successfully materialized, and the `raw\_orders\_no\_invalid\_quantity` asset check returned a "Passed" status. This work reinforced the understanding of asset dependency concepts, cross-container data reading via a shared warehouse, and the implementation of data quality checks within a multi-pipeline Dagster architecture.

CAPSTONE

Capstone — Track B: Cross-Pipeline Analytics

Repository: <https://github.com/nadiraaa123/dagster-workshop-multi>

New pipeline: ``pipeline_reporting``

1. Background

After completing three exercises and studying ``pipeline_ml``, the capstone phase requires building a new, open-ended pipeline. Track B was selected because it focuses on merging data across existing pipelines (``pipeline_products``, ``pipeline_fx``, ``pipeline_ml``) into a single new analytical report, thereby deepening the cross-container read concept learned in Exercise 2.

2. Pipeline Design

``pipeline_reporting`` was built as a fourth independent Dagster container, following the same architectural pattern as the other three pipelines: it has its own ``Dockerfile``, ``requirements.txt``, ``db.py``, and ``main.py``, is registered as a separate code location via ``workspace.yaml``, and is wired into ``docker-compose.yml``.

This pipeline does not interact with any external APIs. Instead, it reads four tables—previously written by other pipelines—directly from ``warehouse_postgresql``: ``orders`` and ``products`` (from ``pipeline_products``), ``exchange_rates`` (from ``pipeline_fx``), and ``order_value_predictions`` (from ``pipeline_ml``). This approach aligns with the workshop's architectural principles: each container remains independent and connects only via the shared warehouse acting as a landing zone, with no direct dependencies between containers.

3. Implementation

Asset ``high_value_orders_report``: combines order and product data to calculate the total value of each order in USD, converts it to EUR using rates from ``exchange_rates``, and filters for orders predicted to be high-value (``predicted_label == 1``) by the model from ``pipeline_ml``. The final result is stored as the ``high_value_orders_report`` table, sorted by the highest EUR value.

Asset check ``high_value_orders_report_has_positive_eur``: validates that no rows in the final report have a zero or negative ``total_eur`` value, serving as a data quality safeguard before the report is deemed suitable for decision-making.

4. Verification

- The `pipeline\_reporting` code location loaded successfully without errors in the Dagster UI.
- "Materialize all" executed end-to-end after `pipeline\_products`, `pipeline\_fx`, and `pipeline\_ml` were materialized first.

Asset name	Code location / Asset group	Status
exchange_rates_table	pipeline_fx default	Materialized 28 Jul, 22:45
high_value_orders_report	pipeline_reporting default	Materialized 29 Jul, 20:56
order_features	pipeline_ml default	Materialized 28 Jul, 22:47
order_value_predictions	pipeline_ml default	Materialized 28 Jul, 22:47
orders_in_eur	pipeline_fx default	Materialized 28 Jul, 22:45
orders_table	pipeline_products default	Materialized 28 Jul, 23:04
products_table	pipeline_products default	Materialized 28 Jul, 23:04
raw_exchange_rates	pipeline_fx default	Materialized 28 Jul, 22:45
raw_orders	pipeline_products default	Materialized 28 Jul, 23:04
raw_products	pipeline_products default	Materialized 28 Jul, 23:04
top_selling_products	pipeline_products default	Materialized 28 Jul, 23:04
trained_model	pipeline_ml default	Materialized 28 Jul, 22:47

- The asset check `high\_value\_orders\_report\_has\_positive\_eur` returned a **Passed** status, with `invalid\_row\_count: 0` out of a total of 19 report rows.

Assets > high_value_orders_report

Overview

Events

Checks

Lineage

Checks (1)

Execute all

high_value_orders_report_has_positive_eur

Filter checks

high_value_ord...s_positive_eur
Succeeded

Overview

Execution history

Automation history

About

No description provided

Latest execution

Evaluation result	Timestamp	Target materialization
Passed	29 Jul, 21:16	29 Jul, 20:56

Metadata

invalid_row_count	0
total_rows	19

Plots

- The unit test (`pipeline\_reporting/tests/test\_main.py`) ran successfully using `pytest`, employing mocks for the database connection and source tables.

```
C:\Users\NADIRA\dagster-workshop-multi\pipeline_reporting>python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.3.5, pluggy-1.6.0 -- C:\Users\NADIRA\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\NADIRA\dagster-workshop-multi\pipeline_reporting
plugins: anyio-4.11.0
collected 1 item

tests/test_main.py::test_high_value_orders_report_filters_and_convert PASSED [100%]

===== 1 passed in 17.94s =====
```

5. Challenges and Solutions

- A YAML parsing error occurred in `workspace.yaml` after manually adding the `pipeline\_reporting` entry, caused by inconsistent indentation. The solution was to rewrite the entire file using a uniform indentation structure (2 spaces per level).
- Docker Compose failed to run because Docker Desktop was not active on the system; this was not a configuration issue, but rather an environmental prerequisite that needed to be running before executing `docker compose up`.

6. Conclusion

Capstone Track B was successfully completed by building `pipeline\_reporting` as the fourth cross-container reporting pipeline in this workshop architecture. All self-check points in `capstone.md` have been met: the pipeline appears as a distinct code location, materialization runs without errors, asset checks pass, pytest passes, it is wired into `docker-compose.yml` and `workspace.yaml`, and it is documented in the forked README using the provided template.